

PROJECT AND TEAM INFORMATION

Project Title

ML-Core: A C++-Based Implementation of Fundamental Machine Learning Algorithms

Student / Team Information

<i>Team Name:</i> <i>Team #</i>	<i>ML Core</i> <i>T170</i>
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	<i>Kotiyal, Shubhangani –</i> <i>2510370294</i> <i>2510370294@geu.ac.in</i> 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	<i>Agarwal, Ishanaya –</i> <i>2510380249</i> <i>2510380249@geu.ac.in</i> 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	<i>Joshi, Harshita –</i> <i>2510380118</i> <i>2510380118@geu.ac.in</i> 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Machine learning is widely used through high-level libraries such as Scikit-learn and TensorFlow. Although these tools are helpful, they can make the internal working of an algorithm appear like a black box. For example, a student can call a function to perform K-Means or SVM without seeing the calculations involved in updating centroids, determining a separating hyperplane, or performing dimensionality reduction. This project takes a more practical approach by implementing Linear Regression, SVM, KNN, K-Means, Hierarchical Clustering, and PCA directly in C++. By implementing these algorithms from scratch, the team will write its own loops, distance calculations, matrix operations, and parameter updates. Classes, objects, and encapsulation will organize each algorithm's data and operations into understandable, reusable components. This approach makes the mathematics behind machine learning models transparent while developing object-oriented programming and computational efficiency skills. Covering regression, classification, clustering, and dimensionality reduction, the goal is not to match the capabilities of existing libraries, but to make machine learning something the team can build, debug, and explain with confidence.

State of the Art / Current solution (1 pt)

At present, much of classical machine learning uses high-level libraries. For example, Scikit-learn offers simple APIs for Linear Regression, SVM, KNN, K-Means, Hierarchical Clustering, and PCA, enabling users to train models and generate results with relatively little code. TensorFlow and PyTorch provide tensor operations and automatic differentiation, with a strong emphasis on deep learning. MATLAB and R also offer statistical and machine learning tools, while graphical applications such as Weka and Orange support experimentation through visual workflows. C++ libraries such as ml-pack, dlib, and Shark provide reusable implementations for a range of machine learning applications. Although these tools offer documentation and access to implementation details, their abstractions and complexity can make it challenging for beginners to follow every mathematical step. Gap Identified: Beginners can benefit from simple C++ programs that explicitly demonstrate the mathematics and sequence of operations behind each algorithm. ML-Core addresses this learning need through readable implementations built from scratch, using classes, objects, and encapsulation to organize the code. This approach will help learners understand each algorithm's calculations while developing their object-oriented programming skills.

Project Goals and Milestones (2 pts)

The main goal of ML-Core is to implement six machine learning algorithms—Linear Regression, SVM, KNN, K-Means, Hierarchical Clustering, and PCA—from scratch in C++ without using external machine learning libraries. The project will help the team understand how these algorithms learn from or identify patterns in data by implementing their mathematical operations step by step. Classes, objects, and encapsulation will organize each algorithm's data and functions. To keep the code understandable and avoid repetition, the team will develop reusable utilities for matrix operations, distance calculations, and CSV file handling. Initial prototypes will use console input before these shared utilities are integrated. The work will proceed through the following stages: *Preparation: Study the mathematics behind each algorithm and translate its steps into program logic. Core Utilities: Develop reusable functions for numerical calculations, distance measurement, matrix operations, and file handling. Algorithm Development: Implement Linear Regression, KNN, SVM, K-Means, Hierarchical Clustering, and PCA as separate C++ classes. Testing and Validation: Test each implementation using small datasets with known results. Compare suitable outputs and evaluation metrics with Python's Scikit-learn, accounting for differences in settings and initialization. Use separate plotting tools to visualize the results. Finalization: Integrate the implemented modules, prepare documentation explaining the mathematics and code, and demonstrate the system and the team's learning outcomes.*

Project Approach (3 pts)

1. How the Code Will Be Built:

We will build the project from scratch in C++, keeping each algorithm in a separate code file with its own class and required functions. Each program will handle its own input, calculations, and output. This will make the algorithms easier to understand, test, debug, and maintain independently.

2. How the Algorithms Will Be Designed Tested:

Before writing code, we will study the mathematics behind each algorithm, such as cost functions, gradients, distances, and eigenvalues where applicable, and convert these theoretical steps into C++ member functions.

3. Testing will happen in three steps:

1. Each program will be tested independently using small datasets with results that can be calculated by hand.
2. Results will be compared with standard implementations such as Scikit-learn using comparable settings.
3. Each program will be checked with different valid and invalid inputs to verify its calculations, predictions, and input handling.

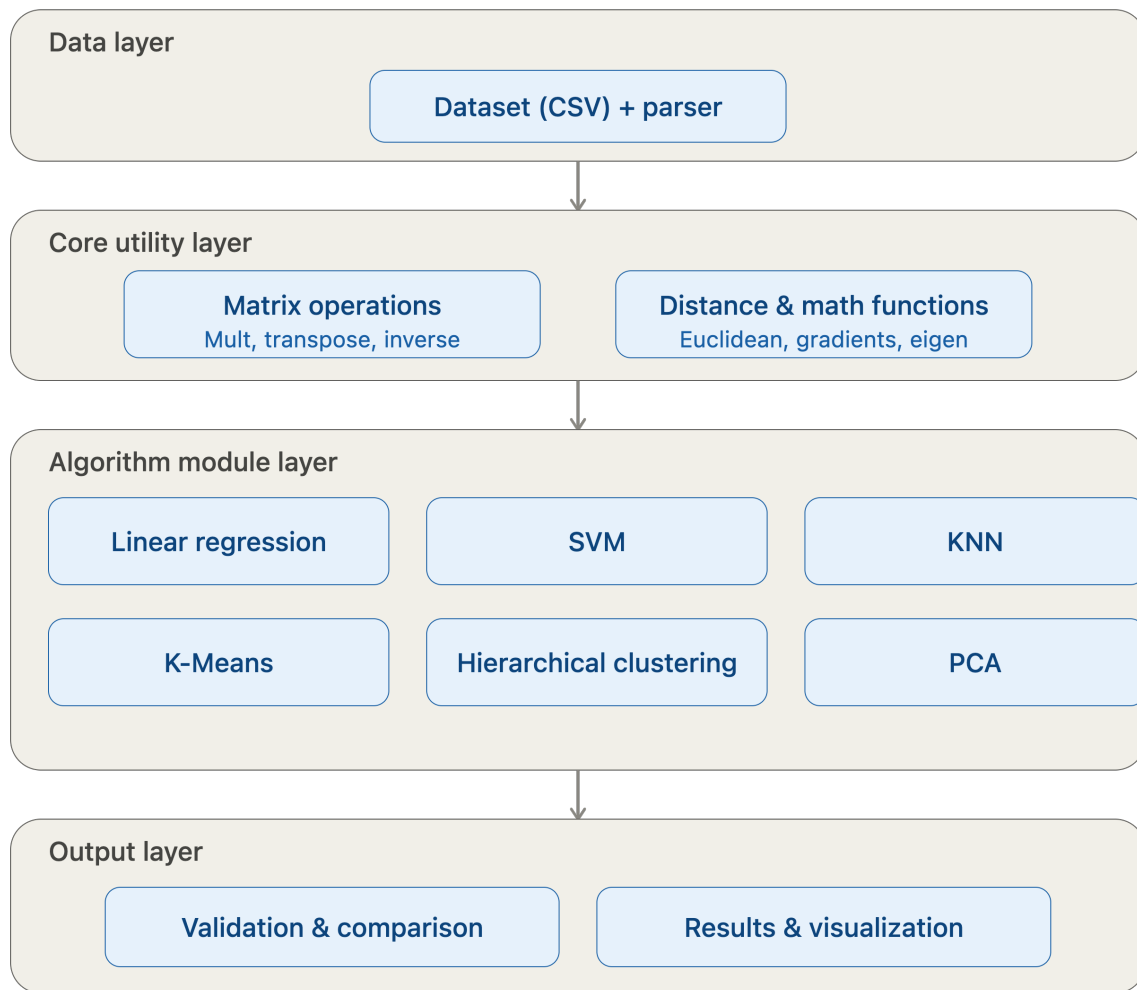


Figure 1: High level Diagram of Architecture

System Architecture (High Level Diagram)(2 pts)

System Architecture

ML-Core follows a modular design in which each algorithm is implemented in a separate C++ code file. Each program contains its own class and functions for input, calculations, and output, and runs independently.

Input Handling: Each program accepts the numerical data and parameters required for its algorithm through console input. Basic checks help identify invalid inputs.

Algorithm Implementation: The project includes planned implementations of Linear Regression, SVM, KNN, K-Means, Hierarchical Clustering, and PCA. Each algorithm's calculations are performed within its own C++ class, without a shared utility layer.

Output Handling: Each program displays its results in the console, such as predicted values, class labels, cluster assignments, or transformed data, depending on the algorithm. Evaluation measures are calculated where implemented, such as Mean Squared Error in Linear Regression.

Data flows independently within each program: the user enters data, the program performs the algorithm's calculations, and the results are displayed in the console. Keeping the algorithms in separate files makes them easier to understand, test, debug, and maintain.

Project Outcome / Deliverables (1 pts)

By the end of this project, the team will deliver:

Working C++ Code: Six core machine learning algorithms—Linear Regression, SVM, KNN, K-Means, Hierarchical Clustering, and PCA—implemented from scratch in C++ without using external machine learning libraries.

Separate Algorithm Programs: Individual code files containing classes and functions for each algorithm's input handling, calculations, and output.

Tested Results: Results checked using small datasets with known answers and compared with Python's Scikit-learn using comparable settings.

Clear Console Output: Readable results showing predicted values, class labels, cluster assignments, transformed data, and evaluation measures where applicable.

Complete Guide: Documentation explaining the mathematics, code structure, execution steps, and test results.

Final Demo: A live demonstration of the individual programs and a report describing the results and lessons learned.

Assumptions

- 1. Data Setup: The programs will work with small numerical datasets within their defined array limits. Data is expected to contain no missing values. Basic input validation will be included, but automatic data cleaning is outside the project's scope.*
- 2. Tools Only: The algorithms will be implemented using standard C++ and headers such as '`<iostream>`' and '`<cmath>`'. No external machine learning libraries will be used in the implementations. Python's Scikit-learn may be used separately for validation.*
- 3. Algorithm Scope: The SVM implementation will focus on linear binary classification. Nonlinear kernels and multiclass SVM are outside the initial scope.*
- 4. Manual Settings: Parameters such as the number of neighbours in KNN, the number of clusters in K-Means, and learning rates where required will be set manually rather than selected automatically.*
- 5. Single Computer: Each program will run independently on a single computer using its CPU. GPU processing and distributed computing are outside the project's scope.*
- 6. Focus on Learning: The project is designed for educational purposes, prioritizing understandable code, mathematical clarity, and correctness rather than large-scale production use.*

References

The following resources and references were used while planning the project:

1. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. <https://link.springer.com/book/9780387310732>
2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. <https://hastie.su.domains/ElemStatLearn/>
3. Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
4. Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press. <https://probml.github.io/pml-book/>
5. Cortes, C., & Vapnik, V. (1995). Support-Vector Networks. *Machine Learning*, 20(3), 273–297. <https://doi.org/10.1007/BF00994018>
6. Cover, T., & Hart, P. (1967). Nearest Neighbor Pattern Classification. *IEEE Transactions on Information Theory*, 13(1), 21–27. <https://doi.org/10.1109/TIT.1967.1053964>
7. MacQueen, J. (1967). Some Methods for Classification and Analysis of Multivariate Observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297. <https://projecteuclid.org/ebooks/berkeley-symposium-on-mathematical-statistics-and-probability/Proceedings-of-the-Fifth-Berkeley-Symposium-on-Mathematical-Statistics-and-probability/chapter/Some-methods-for-classification-and-analysis-of-multivariate-observations/bsmsp/1200512992>
8. Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer. <https://link.springer.com/book/10.1007/b98835>
9. Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
10. Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. (2007). *Numerical Recipes: The Art of Scientific Computing* (3rd ed.). Cambridge University Press. <http://numerical.recipes/>
11. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
12. Ng, A. CS229: Machine Learning Course Notes. Stanford University. <https://cs229.stanford.edu/>
13. GeeksforGeeks. Machine Learning Algorithms. <https://www.geeksforgeeks.org/machine-learning/>
14. Scikit-learn Documentation. <https://scikit-learn.org/stable/documentation.html>